DÍA 1 • FUNDAMENTOS DE CONTENEDORES

DOCKER FUNDAMENTALS

Módulo 1: Introducción, Arquitectura del Motor, Seguridad e Imágenes
Jornada 1 de 4 (5 Horas Lectivas)

El Reto del Despliegue: Del Monolito a la Matriz Matricial

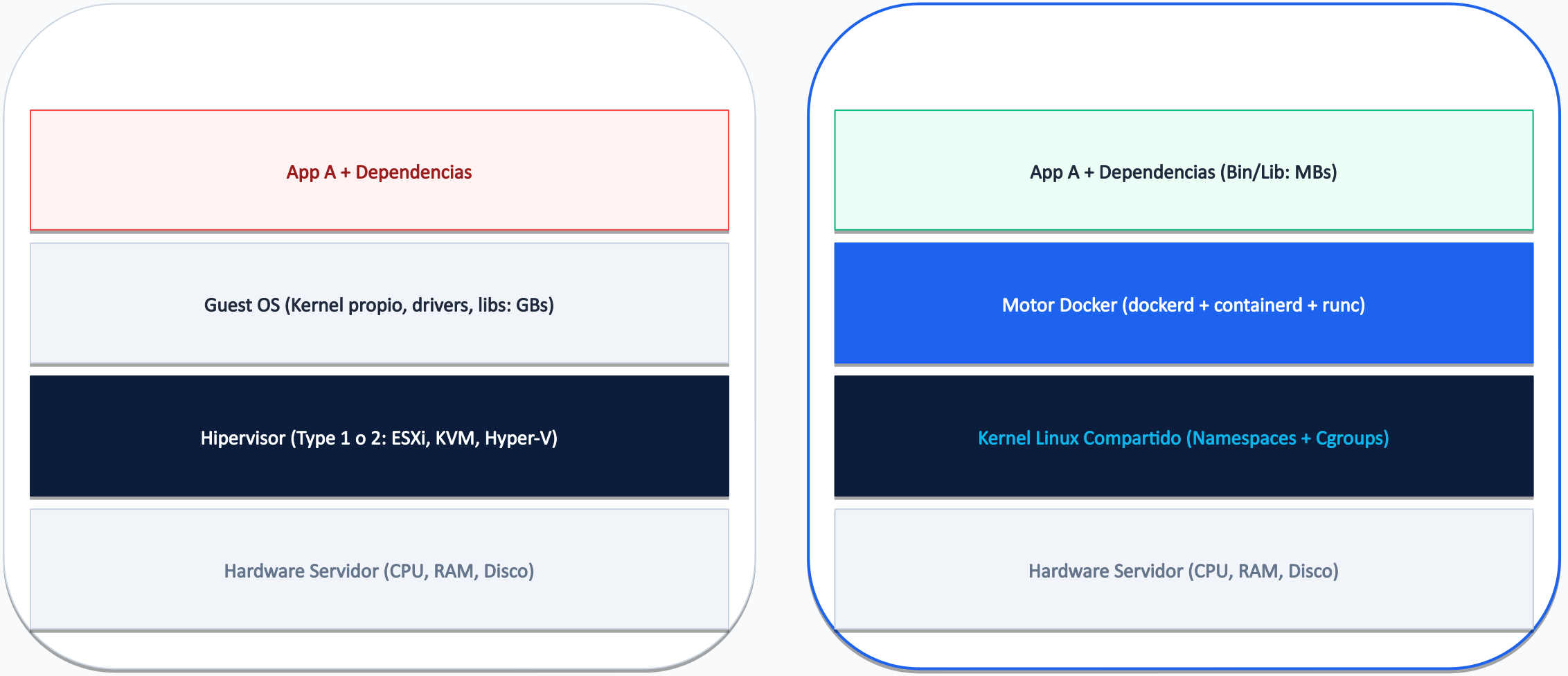
La Matriz del Infierno (N x M)

- **Diversidad de Lenguajes:** Node.js, Python, Java, Go, .NET con diferentes versiones de runtime en el mismo servidor.
- **Dependencias de Sistema:** Librerías compartidas (glibc, OpenSSL) que generan conflictos catastróficos entre servicios.
- **Entornos Heterogéneos:** Discrepancias constantes entre portátil local, servidor de pruebas (QA) y clúster de producción.
- **Deriva de Configuración:** Modificaciones manuales no documentadas en máquinas virtuales ('servidores copo de nieve').

La Solución de Contenedores

- **Inmutabilidad:** Empaquetar la aplicación junto con todo su entorno exacto de ejecución en un artefacto autocontenido.
- **Portabilidad Absoluta:** La garantía del contenedor: si arranca en el equipo del desarrollador, se ejecutará idénticamente en la nube.
- **Arranque en Milisegundos:** Al no tener que arrancar un sistema operativo completo, un contenedor inicia como un proceso normal.
- **Densidad de Carga:** Ejecución de cientos de microservicios en el mismo hardware sin la sobrecarga de RAM de múltiples VMs.

Máquinas Virtuales vs Contenedores: Arquitectura Interna



¿Qué es un Contenedor? El Secreto del Kernel

La Verdad Técnica

- **No existe la 'entidad contenedor':** En el código fuente del kernel de Linux no hay ninguna estructura de datos llamada 'container'.
- **Un proceso ordinario:** Un contenedor es un proceso estándar de Linux (`task_struct`) al que se le han aplicado restricciones de visibilidad y recursos.
- **Los Tres Pilares del Kernel:** Namespaces (visión aislada), Cgroups (cuotas de recursos) y LSM/Capabilities (seguridad).
- **Coste Cero de Virtualización:** La CPU ejecuta las instrucciones del contenedor nativamente en Ring 3 (espacio de usuario).

$ Comprobación de Procesos

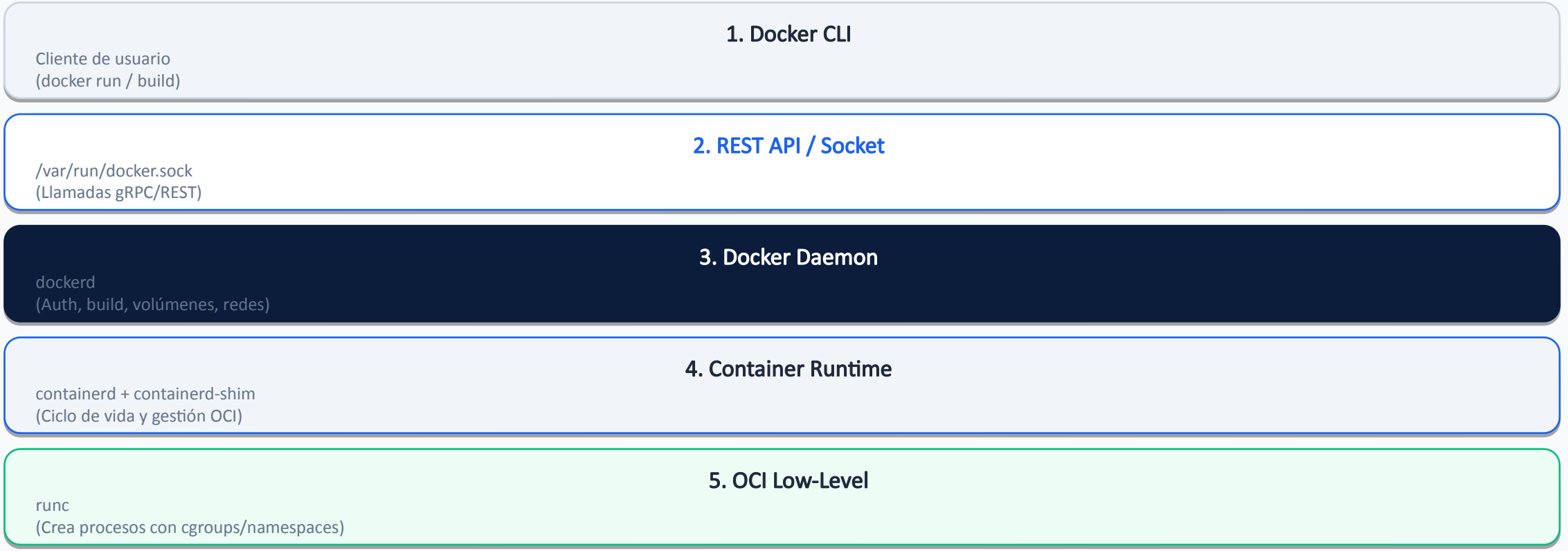
```
# Ver el proceso del contenedor desde el Host
$ docker run -d --name web-demo nginx
7f8a9b2c...

# Inspeccionar el PID en el host
$ docker inspect -f '{{.State.Pid}}' web-demo
42150

# Comprobar que en el host es un proceso normal
$ ps -ef | grep 42150
root 42150 42100 0 10:15 ? 00:00:00 nginx: master

# Comprobar que dentro del contenedor se ve como PID 1
$ docker exec web-demo ps -ef
UID  PID  PPID  C STIME TTY   TIME CMD
root   1     0  0 10:15 ? 00:00:00 nginx: master
```

Arquitectura del Motor Docker: Desacoplamiento OCI



Docker-Engine vs Docker.io: Paquetes y Distribuciones

Docker-CE / Docker-Engine (Oficial)

• **Origen:** Repositorios oficiales mantenidos directamente por Docker Inc.

• **Versiones Actualizadas:** Incluye las últimas mejoras de runtime, compatibilidad OCI, Docker Buildx y Compose v2 integrado.

• **Paquetes Clave:** `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin`, `docker-compose-plugin`.

• **Uso Recomendado:** Entornos de producción empresariales y servidores de desarrollo profesional.

Docker.io (Distribuciones Linux)

• **Origen:** Mantenido por empaquetadores de distribuciones (Debian, Ubuntu) en sus repositorios `apt` estándar.

• **Versiones Desfasadas:** Suele incluir versiones antiguas congeladas para asegurar estabilidad con la versión de la distribución.

• **Ausencia de Plugins Modernos:** A menudo carece de Buildx moderno o el comando `docker compose` unificado.

• **Riesgos de Compatibilidad:** Problemas con sintaxis de Compose moderna o drivers de almacenamiento recientes.

Filosofía de Seguridad: Defensa en Profundidad

NAMESPACES (¿Qué puede ver?)

- PID: Árbol de procesos privado (PID 1)
- NET: Interfaces de red y puertos propios
- MNT: Puntos de montaje aislados
- IPC: Memoria compartida y colas
- UTS: Hostname y dominio independiente
- USER: Mapeo de UIDs/GIDs (rootless)

CGROUPS & SEC (¿Cuánto puede usar?)

- CPU: Límites (--cpus, shares, quotas)
- Memoria: Topes y prevención OOM Killer
- Block I/O: Ancho de banda de disco
- PIDs: Límite contra fork-bombs
- Capabilities: Restricción de llamadas root
- Seccomp & AppArmor: Filtro de syscalls

Linux Namespaces: Creando la Ilusión de Máquina Privada

Los 6 Namespaces Fundamentales del Kernel

• **PID Namespace:** Proporciona un árbol de procesos privado. El proceso de entrada del contenedor se convierte en el PID 1, ajeno al resto de procesos del host.

• **NET Namespace:** Proporciona una pila de red completa (loopback privada, interfaz veth propia, tabla de enrutamiento y reglas iptables individuales).

• **MNT (Mount) Namespace:** Aísla la jerarquía del sistema de archivos. Cada contenedor ve su propio rootfs `/` generado por el Union File System.

• **IPC Namespace:** Impide la comunicación por memoria compartida (POSIX/SysV) o semáforos entre procesos de contenedores distintos.

• **UTS Namespace:** Permite que el contenedor tenga su propio hostname y nombre de dominio sin alterar el nombre del servidor anfitrión.

• **USER Namespace:** Asigna el usuario UID 0 (root) dentro del contenedor a un UID no privilegiado (ej. 100000) en el host (Base de Rootless Docker).

Control Groups (Cgroups): Límites y Medición

Protección de Recursos en el Host

- **Evitar Ataques DoS / Vecino Ruidoso:** Un contenedor con un bucle infinito o fuga de memoria no debe colapsar el servidor entero.
- **Límites de Memoria (`--memory`):** Control estricto de RAM. Si el contenedor excede su límite, el kernel OOM Killer termina el proceso del contenedor, no el host.
- **Cuotas de CPU (`--cpus`):** Asignación de periodos de CPU mediante el CFS (Completely Fair Scheduler) de Linux.
- **Cgroups v2 (Moderno):** Jerarquía unificada de control en `/sys/fs/cgroup/`, mejor gestión de memoria y presión de E/S (PSI).

$ Aplicación de Límites

```
# Limitar contenedor a 512MB de RAM y 1.5 CPUs
$ docker run -d --name app-limitada \
    --memory="512m" \
    --cpus="1.5" \
    --pids-limit=100 \
    nginx

# Monitorizar métricas en tiempo real
$ docker stats app-limitada
CONTAINER ID   NAME           CPU %   MEM USAGE / LIMIT
8a1b2c3d4e     app-limitada   0.05%   4.2MiB / 512MiB

# Inspeccionar cgroups directamente en Linux
$ cat /sys/fs/cgroup/docker/<ID>/memory.max
536870912
```

Capabilities de Linux y Modo Rootless

El Principio de Mínimo Privilegio

- **Fraccionando el poder de Root:** Linux divide los privilegios de superusuario en ~40 'capabilities' independientes (ej. `CAP_NET_ADMIN`, `CAP_SYS_ADMIN`, `CAP_CHOWN`).
- **Capabilities por Defecto en Docker:** Docker arranca contenedores eliminando capabilities peligrosas (`CAP_SYS_ADMIN`, `CAP_SYS_RAWIO`).
- **Antipatrón `--privileged`:** Proporciona acceso ilimitado a todos los dispositivos del host. ¡Prohibido en producción!
- **Rootless Docker:** Ejecución de dockerd y contenedores completamente bajo una cuenta de usuario sin privilegios en el host.

$ Gestión de Capabilities

```
# Buenas prácticas: Despojar todas las capabilities
# y conceder únicamente las indispensables
$ docker run -d --name web-secure \
    --cap-drop=ALL \
    --cap-add=NET_BIND_SERVICE \
    nginx

# Comprobar que no se pueden modificar interfaces
$ docker exec web-secure ip link add dummy0 type
dummy
RTNETLINK answers: Operation not permitted

# Peligro extremo (Antipatrón):
$ docker run --privileged ... # NUNCA EN PRODUCCIÓN
```

¿Qué es una Imagen en Docker? La Plantilla Inmutable

Definición y Propiedades

• **Plantilla de Solo Lectura:** Archivo binario empaquetado que contiene el código, runtime, librerías del sistema y variables de entorno necesarias para ejecutar una aplicación.

• **Inmutabilidad Estricta:** Una vez creada, una imagen jamás se modifica; para hacer cambios se genera una nueva versión.

• **Identificador Criptográfico (Digest):** Cada capa e imagen se identifica por su hash SHA-256 (`sha256:7b9a...`), garantizando integridad frente a manipulaciones.

• **Especificación OCI Image Format:** Estándar abierto gobernado por la Linux Foundation, compatible con Podman, CRI-O, Containerd y Docker.

Anatomía de una Referencia de Imagen

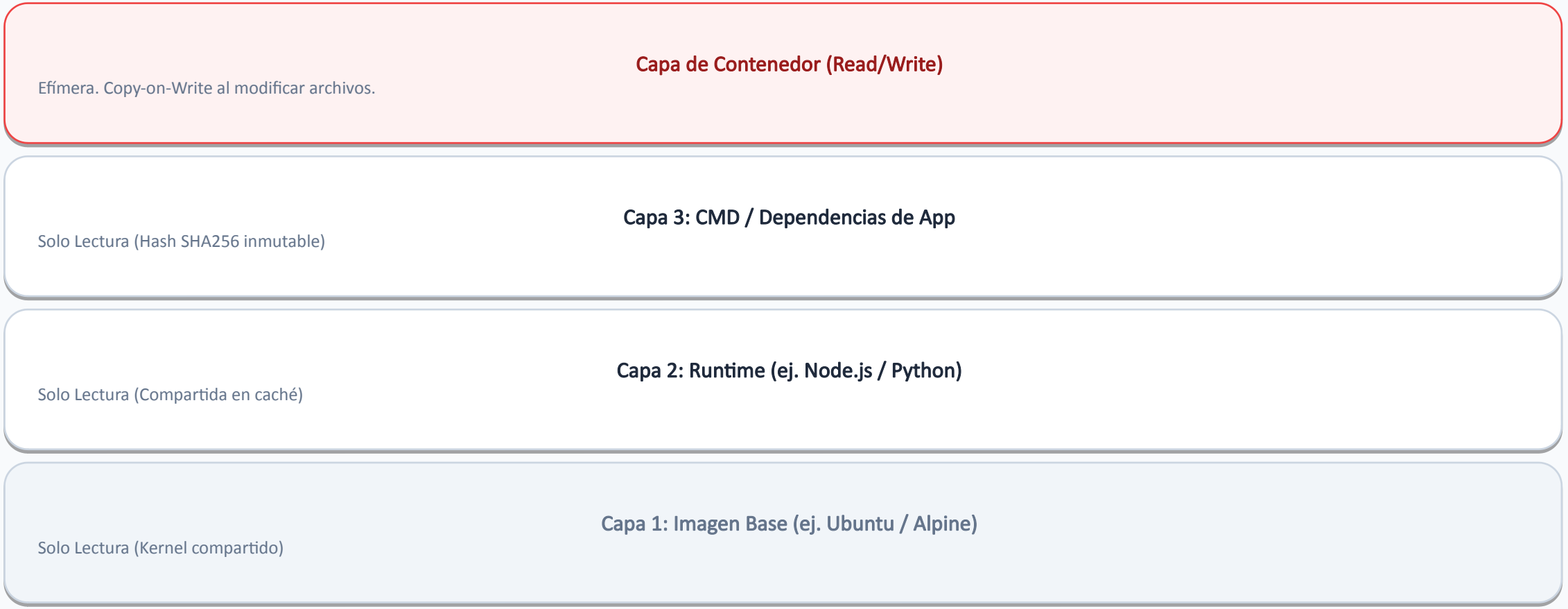
• **Estructura Estándar:** `[registry]/[namespace]/[repositorio]:[tag]`.

• **Ejemplo Completo:** `docker.io/library/nginx:1.25-alpine`.

• **Valores por Defecto:** Si no se indica registry, se asume `docker.io` (Docker Hub); si no se indica tag, se asume `:latest`.

• **Peligro del tag `:** latest`: No significa necesariamente 'última versión estable', sino la etiqueta por defecto asignada al último build. En producción, fijar siempre versiones semánticas (ej. `:1.25.3-alpine`).

Características de una Imagen: Capas y Caché



¿Qué es un Contenedor? La Instancia Viva

La Relación Clase - Objeto

• **Analogía de Programación:** La imagen es la **Clase** (definición inmutable); el contenedor es el **Objeto** (instancia viva en memoria).

• **Añadido de Capa R/W:** Al instanciar un contenedor, Docker añade una capa de lectura/escritura muy fina en la parte superior.

• **Mecanismo Copy-on-Write (CoW):** Si el contenedor modifica un archivo de la imagen, el driver copia el archivo a la capa superior antes de modificarlo.

• **Naturaleza Efímera:** Al eliminar el contenedor (`docker rm`), su capa R/W se destruye por completo; la imagen original permanece intacta.

$ Demostración de Copy-on-Write

```
# 1. Instanciar un contenedor a partir de Ubuntu
$ docker run -it --name prueba-cow ubuntu bash

# 2. Modificar un archivo del sistema
root@abc123:/# echo "modificado" >> /etc/issue

# 3. Salir y ver los cambios en la capa R/W
$ docker diff prueba-cow
C /etc
C /etc/issue    # C = Changed (Copiado a R/W)
A /root/.bash_history  # A = Added

# 4. La imagen base jamás se altera
$ docker run --rm ubuntu cat /etc/issue
```

Ciclo de Vida y Máquina de Estados del Contenedor

Estados Principales

• **Created (`docker create`):** Se prepara el sistema de archivos y configuración, pero el proceso aún no arranca.

• **Running (`docker run / start`):** El proceso principal (PID 1) está activo en el kernel consumiendo CPU y memoria.

• **Paused (`docker pause`):** Se congelan los procesos usando el cgroup freezer sin liberar memoria RAM.

• **Exited / Stopped (`docker stop`):** Se envía señal SIGTERM (10s de gracia) y luego SIGKILL. El proceso finaliza.

• **Dead:** Estado de error irrecuperable por bloqueo de kernel o fallo en desmontaje de volúmenes.

Gestión de Señales UNIX en PID 1

• **Parada Limpia (`SIGTERM` - señal 15):** Docker notifica al contenedor que debe cerrar conexiones, guardar estado y terminar.

• **Parada Forzosa (`SIGKILL` - señal 9):** Si no finaliza en el periodo de gracia (por defecto 10s), el kernel mata el proceso.

• **El Problema del Manejador en PID 1:** En Linux, el PID 1 ignora por defecto señales si la aplicación no registra un handler explícito.

• **Herramienta `--init`:** Inyecta un gestor init ultraligero (`tini`) para reenviar señales y recolectar procesos zombies.

Comandos Esenciales de Operación en el Aula

$ Comandos de Contenedor

```
# Ejecutar en segundo plano con puerto y nombre
$ docker run -d --name mi-nginx -p 8080:80 nginx

# Listar contenedores activos y todos (incluidos parados)
$ docker ps
$ docker ps -a

# Ver logs en tiempo real (seguimiento continuo)
$ docker logs -f --tail 50 mi-nginx

# Ejecutar un comando interactivo dentro del contenedor
$ docker exec -it mi-nginx bash

# Inspeccionar configuración completa en JSON
$ docker inspect mi-nginx | grep -i IPAddress
```

$ Gestión de Imágenes y Limpieza

```
# Descargar una imagen desde Docker Hub
$ docker pull redis:7-alpine

# Listar imágenes locales y su tamaño
$ docker images

# Detener y eliminar un contenedor
$ docker stop mi-nginx
$ docker rm mi-nginx

# Limpieza global del sistema (espacio en disco)
$ docker system df
$ docker system prune -af --volumes
# Elimina contenedores parados, redes y capas huérfanas
```

Automatizando la Creación: El Dockerfile

Infraestructura como Código (IaC)

- **Fichero Declarativo:** Archivo de texto plano con instrucciones secuenciales que definen paso a paso cómo construir una imagen.
- **Reproducibilidad Total:** El Dockerfile se almacena en Git junto al código fuente de la aplicación, permitiendo auditoría y control de versiones.
- **Construcción Paso a Paso:** Cada instrucción (`RUN`, `COPY`, `ADD`) genera una nueva capa intermedia en el sistema de almacenamiento.
- **Motor BuildKit:** El compilador moderno de Docker que ejecuta análisis de dependencias y paraleliza capas independientes.

$ Estructura de un Dockerfile

```
# Dockerfile de ejemplo para aplicación Node.js
FROM node:20-alpine

# Definir directorio de trabajo en el contenedor
WORKDIR /usr/src/app

# Copiar dependencias y compilar
COPY package*.json ./
RUN npm ci --only=production

# Copiar código fuente
COPY . .

# Exponer puerto y usuario sin privilegios
USER node
EXPOSE 3000

# Comando de arranque
CMD ["node", "server.js"]
```

Directivas Fundamentales de Dockerfile

Directivas de Entorno y Sistema

• **`FROM <imagen>:** <tag>`: Establece la imagen base. Debe ser la primera instrucción obligatoria.

• **`WORKDIR /ruta`:** Establece el directorio de trabajo para todas las instrucciones siguientes (evita usar múltiples `cd`).

• **`COPY <origen> <destino>`:** Copia archivos y carpetas locales del host al sistema de archivos de la imagen.

• **`ENV CLAVE=VALOR`:** Define variables de entorno que persisten tanto durante el build como al arrancar el contenedor.

• **`EXPOSE <puerto>`:** Documenta qué puerto escucha el servicio (metadato informativo, no abre el puerto en el host).

CMD vs ENTRYPOINT

• **`RUN <comando>`:** Ejecuta comandos durante la fase de construcción (ej. `apt-get install`) y crea una nueva capa.

• **`ENTRYPOINT ["ejecutable"]`:** Define el comando fijo inamovible que siempre se ejecutará al iniciar el contenedor.

• **`CMD ["param1", "param2"]`:** Parámetros por defecto que se pasan a ENTRYPOINT o comando por defecto anulable.

• **Sintaxis Exec (Recomendada):** Siempre usar corchetes y comillas dobles `["prog", "arg"]` para que se ejecute directamente como PID 1 sin invocar una subshell `/bin/sh -c`.

Buenas Prácticas: Optimización de Capas y Caché

Aprovechamiento de la Caché de Build

• **De lo Menos Cambiante a lo Más Frecuente:** Colocar las instrucciones que rara vez cambian (OS, runtimes, dependencias) al inicio del Dockerfile.

• **Invalidación en Cascada:** En cuanto una capa cambia (ej. un `COPY . .`), todas las capas posteriores invalidan su caché y se reconstruyen.

• **Uso de `.dockerignore`:** Excluir carpetas pesadas (`node_modules`, `.git`, temporales) para no inflar el build context ni invalidar caché accidentalmente.

• **Encadenar Comandos RUN:** Concatenar `RUN apt-get update && apt-get install -y ... && rm -rf /var/lib/apt/lists/*` en una única capa para no almacenar caché de paquetes.

$ Comparativa de Patrones

```
# ANTIPATRÓN (Invalida caché en cada cambio de código):
COPY . .
RUN npm install  # ¡Se ejecuta siempre, muy lento!

# BUENA PRÁCTICA (Aprovecha caché de dependencias):
COPY package*.json ./
RUN npm ci       # Solo se ejecuta si cambia package.json
COPY . .         # Si solo cambia el código, usa caché previa

# Ejemplo de limpieza de paquetes en una sola capa:
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    ca-certificates \
 && rm -rf /var/lib/apt/lists/*
```

Multi-Stage Builds: Seguridad y Mínimo Tamaño

¿Por qué Multi-Stage?

- **Separar Compilación de Ejecución:** Un entorno de build necesita compiladores (gcc, go, maven, npm, SDKs completos) que pesan gigabytes.
- **Producción Limpia:** En producción solo necesitamos el artefacto binario y el runtime mínimo indispensable.
- **Reducción Drástica de Vulnerabilidades:** Al no incluir compiladores ni herramientas de build en la imagen final, reducimos los CVEs al mínimo.
- **Imágenes Distroless y Alpine:** Crear imágenes finales con tamaños de 5MB a 30MB en lugar de 1.5GB.

$ Ejemplo Multi-Stage

```
# ETAPA 1: Compilación (Builder)
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o mi-app

# ETAPA 2: Runtime de Producción (Final)
FROM alpine:3.19
WORKDIR /app
# Copiar ÚNICAMENTE el binario de la etapa anterior
COPY --from=builder /app/mi-app .
USER guest
EXPOSE 8080
CMD ["./mi-app"]
# Resultado: De una imagen de 800MB a una de solo 12MB
```

Docker y la Arquitectura de Microservicios

La Regla de Oro: Un Proceso por Contenedor

• **Desacoplamiento Funcional:** No intentar meter Apache, PHP y MySQL en el mismo contenedor como si fuera un servidor VPS tradicional.

• **Ciclos de Vida Independientes:** Cada microservicio debe poder escalarse, actualizarse o reiniciarse sin afectar al resto.

• **Facilidad de Diagnóstico:** Cuando un contenedor falla, sus logs (`docker logs`) reflejan únicamente la salida de ese servicio específico.

• **Superficie de Fallo Acotada:** Si el microservicio de pagos tiene un memory leak, el microservicio de catálogo de productos continúa funcionando intacto.

Retos Emergentes

• **Comunicación entre Servicios:** ¿Cómo hablan entre sí los microservicios sin acoplar direcciones IP fijas? (Respuesta: Redes en Módulo 2).

• **Persistencia de Datos:** Si los contenedores son efímeros, ¿dónde guardamos los datos de base de datos? (Respuesta: Volúmenes en Módulo 3).

• **Despliegue Multi-Contenedor:** Arrancar 15 servicios con `docker run` individual es inviable. Se requiere orquestación (Docker Compose y Kubernetes en Módulo 5).

• **Observabilidad:** Monitorización centralizada de logs, métricas y trazas distribuidas.

Laboratorio 1: Ciclo de Vida y Creación de Imágenes

Objetivos y Pasos del Ejercicio Práctico

- **Paso 1:** Verificar el entorno local y versiones (`docker version`, `docker info`).
- **Paso 2:** Descargar e interactuar con imágenes base (`docker run -it alpine sh`, explorar `uname -a` y procesos).
- **Paso 3:** Crear una aplicación web sencilla en Python/Flask o Node.js y redactar su `Dockerfile` optimizado.
- **Paso 4:** Construir la imagen con tag semántico (`docker build -t mi-app:1.0 .`).
- **Paso 5:** Arrancar el contenedor publicando el puerto hacia el host (`docker run -d -p 8080:5000 --name web mi-app:1.0`).
- **Paso 6:** Probar la aplicación con `curl http://localhost:8080`, inspeccionar logs con `docker logs -f` y comprobar el consumo de recursos con `docker stats`.
- **Paso 7:** Detener y limpiar recursos (`docker stop`, `docker rm`).

RESUMEN DE LA JORNADA 1

Conceptos consolidados: Kernel de Linux, Namespaces, Cgroups, Imágenes OCI, Dockerfile y Ciclo de Vida.
Próxima Jornada: Módulo 2 (Redes a fondo) y Módulo 3 (Persistencia y Volúmenes).

REDES Y PERSISTENCIA

Módulo 2: Redes en Docker • Módulo 3: Volúmenes y Persistencia
Jornada 2 de 4 (5 Horas Lectivas)

Anatomía de la Red en Docker: veth pairs y Linux Bridge

El Puente Virtual y los Pares Veth

- **Aislamiento de Red:** Cada contenedor tiene su propia pila de red (su propia loopback `127.0.0.1`, interfaces y tabla de rutas).
- **Pares de Interfaces Virtuales (`veth`):** Docker crea un par de interfaces conectadas entre sí como si fueran un cable Ethernet virtual.
- **Extremo A (Contenedor):** Reside dentro del Network Namespace del contenedor y se renombra a `eth0`.
- **Extremo B (Host):** Reside en el host (ej. `veth4a91...`) y se conecta al conmutador virtual bridge (`docker0`).
- **Puente `docker0`:** Actúa como un switch Ethernet de nivel 2 (L2) que reenvía tramas entre contenedores del mismo host.

$ Inspección de Interfaces de Red

```
# Comprobar el switch virtual docker0 en el host
$ ip addr show docker0
inet 172.17.0.1/16 scope global docker0

# Arrancar un contenedor e inspeccionar su veth
$ docker run -d --name test-net nginx:alpine

# Listar los enlaces veth conectados al bridge
$ ip link show type veth
veth7a21@if8: <BROADCAST,MULTICAST,UP,LOWER_UP> ...

# Ver la IP privada asignada al contenedor
$ docker inspect -f
'{{.NetworkSettings.IPAddress}}' test-net
172.17.0.2
```

La Red Bridge por Defecto y sus Limitaciones

Características del Bridge por Defecto

• **Comportamiento Out-of-the-Box:** Si no se especifica `--network`, los contenedores se conectan a `bridge`.

• **Subred Típica:** Habitualmente `172.17.0.0/16`. El host toma la IP `172.17.0.1` como gateway predeterminado.

• **Acceso a Internet Saliente:** Docker configura reglas `iptables MASQUERADE` (NAT) para que los contenedores puedan salir a Internet a través de la tarjeta física del host.

• **Asignación DHCP Interna:** Docker gestiona internamente la asignación secuencial de IPs dentro del rango.

La Gran Limitación: Sin Resolución DNS

• **¡No hay DNS Integrado!:** En la red bridge por defecto, dos contenedores NO pueden resolverse por su nombre (ej. `ping backend` fallará).

• **Dependencia Frágil de IPs:** Si un contenedor se reinicia y cambia de `172.17.0.2` a `172.17.0.3`, la aplicación se rompe.

• **Legacy Links (`--link`):** El mecanismo antiguo de inyectar variables en `/etc/hosts` está formalmente marcado como DEPRECATED.

• **Solución Obligatoria:** Crear redes personalizadas (*User-Defined Bridges*).

Redes Personalizadas: Resolución DNS Automática

Beneficios de User-Defined Bridges

- **Servidor DNS Embebido (`127.0.0.11`):** Docker levanta un daemon DNS interno que resuelve automáticamente el nombre del contenedor a su IP actual.
- **Aislamiento por Diseño:** Contenedores en redes distintas no pueden comunicarse entre sí, incrementando la seguridad.
- **Conexión en Caliente:** Posibilidad de conectar o desconectar contenedores de una red en tiempo de ejecución sin reiniciarlos (`docker network connect`).
- **Alias de Red:** Posibilidad de asignar nombres secundarios a un contenedor (ej. `--network-alias db-principal`).

$ Ejemplo de DNS Automático

```
# 1. Crear red personalizada para la aplicación
$ docker network create red-interna

# 2. Arrancar base de datos en dicha red
$ docker run -d --name mi-postgres \
    --network red-interna \
    -e POSTGRES_PASSWORD=secreto postgres:alpine

# 3. Arrancar API en la misma red y resolver por
nombre
$ docker run --rm --network red-interna alpine \
    ping -c 2 mi-postgres
PING mi-postgres (172.19.0.2): 56 data bytes
64 bytes from 172.19.0.2: seq=0 ttl=64 time=0.082
ms

# ¡Resolución DNS automática funcionando!
```

Mapeo de Puertos: Abriendo el Contenedor al Mundo

La Diferencia entre -p y -P

• **Mapeo Explícito (`-p [host_port]:** [container_port]`): Vincula un puerto específico del host a un puerto del contenedor (ej. `-p 80:80` o `-p 127.0.0.1:8080:80`).

• **Mapeo Efímero (`-P` mayúscula):** Mapea todos los puertos definidos en las directivas `EXPOSE` del Dockerfile a puertos aleatorios del host (rango 32768-60999).

• **Seguridad:** Si omites la IP del host (`-p 8080:80`), Docker escuchará en `0.0.0.0` (todas las interfaces, incluyendo Internet público). Para entornos de desarrollo local, fijar `-p 127.0.0.1:8080:80`.

• **Mapeo de Protocolos:** Por defecto mapea TCP. Para UDP se debe especificar explícitamente `-p 53:53/udp`.

$ Mecanismo Interno: iptables DNAT

```
# ¿Cómo implementa Docker el mapeo en Linux?
# Inserta reglas en la tabla NAT de iptables:
$ iptables -t nat -L DOCKER -n -v

Chain DOCKER (2 references)
pkts bytes target  prot opt in  out  source
destination
  15   900 DNAT    tcp  --  *   *    0.0.0.0
0.0.0.0 \
  tcp dpt:8080 to:172.18.0.2:80

# Demonio docker-proxy:
# Para tráfico loopback, Docker utiliza un proceso
# llamado docker-proxy para reenviar conexiones
TCP.
```

Modo de Red Host: Rendimiento Máximo sin Aislamiento

Características de `--network host`

• **Sin Network Namespace:** El contenedor comparte directamente la pila de red del sistema operativo anfitrión.

• **Cero Penalización de NAT:** No hay `veth`, no hay puente `docker0` ni reglas de `iptables`. Máximo throughput y menor latencia posible.

• **Mapeo Innecesario:** Los flags `-p` y `-P` se ignoran. Si la aplicación escucha en el puerto 80, estará escuchando directamente en el puerto 80 del host.

• **Limitación Crítica en Docker Desktop:** En Windows y macOS, `--network host` se conecta a la máquina virtual oculta de Linux, no al Windows/macOS real.

Riesgos y Casos de Uso

• **Colisión de Puertos:** No es posible ejecutar dos contenedores en modo host escuchando en el mismo puerto (el segundo fallará con `Address already in use`).

• **Pérdida de Aislamiento:** El contenedor tiene acceso a todas las interfaces locales, incluyendo `localhost` del host y servicios sensibles.

• **Casos de Uso Válidos:** Sistemas de monitorización de red (Prometheus node-exporter), proxies de altísima concurrencia, transmisiones multimedia RTP/WebRTC en tiempo real.

• **Recomendación General:** Evitar en producción ordinaria salvo necesidad estricta de rendimiento extremo.

Modo de Red None: Aislamiento Total Offline

Aislamiento Estricto (`--network none`)

• **Desconexión Total:** El contenedor dispone de su propio Network Namespace, pero únicamente se configura la interfaz de bucle invertido (`lo` / `127.0.0.1`).

• **Sin Puertos Externos:** Imposible enviar o recibir paquetes fuera del contenedor; sin acceso a Internet ni a otros contenedores.

• **Seguridad Máxima contra Filtraciones:** Si el código es malicioso o vulnerable, no puede descargar payloads externos ni exfiltrar secretos por red.

• **Configuración Manual Avanzada:** Permite a herramientas externas de orquestación inyectar interfaces de red personalizadas a posteriori.

Casos de Uso Típicos

• **Procesamiento Batch Offline:** Procesamiento de archivos confidenciales, transcripciones médicas o cálculo numérico sin salida a Internet.

• **Generación y Firma Criptográfica:** Módulos HSM de software, generación de claves PGP o carteras frías (air-gapped containers).

• **Ejecución de Código No Confiable:** Sandboxing de scripts subidos por usuarios finales sin riesgo de ataques hacia la infraestructura interna.

• **Inspección Forense:** Análisis de imágenes sospechosas sin permitir comunicación con centros de comando y control (C2).

Redes Overlay: Conectando Contenedores en Varios Hosts

Superposición sobre Redes Físicas (VXLAN)

• **El Reto Multi-Host:** La red bridge solo funciona en un único servidor. ¿Cómo comunicamos contenedores en servidores físicos diferentes?

• **Encapsulación VXLAN:** La red overlay encapsula las tramas Ethernet L2 de los contenedores dentro de paquetes UDP estándar (puerto 4789).

• **Criptografía Opcional:** Soporte nativo para cifrado IPsec en vuelo (`--opt encrypted`), protegiendo el tráfico entre nodos en la nube pública.

• **Descubrimiento Distribuido:** Docker propaga la tabla de enrutamiento y nombres de contenedores entre todos los nodos del clúster.

Ámbitos de Aplicación

• **Docker Swarm:** El driver `overlay` es el controlador nativo y estándar para desplegar servicios multi-nodo en Swarm.

• **Kubernetes (K8s):** Utiliza plugins CNI basados en tecnologías de overlay similares (Calico, Flannel VXLAN, Cilium con eBPF).

• **Independencia de la Infraestructura:** Los contenedores se comunican en una subred privada virtual (ej. `10.0.0.0/24`) con total abstracción de los switches físicos.

• **Plan de Control Distribuido:** Utiliza protocolos gossip y consensus (Raft) para sincronizar IPs y estado entre los servidores.

Macvlan y Complementos de Red (Network Plugins)

Driver Macvlan: IP en la LAN Física

• **Presencia Física Directa:** Cada contenedor obtiene una dirección MAC única e independiente y una IP asignada directamente por el router físico de la empresa.

• **Sin NAT ni Port Mapping:** El contenedor aparece como un servidor físico más conectado al switch de la red de la oficina o del centro de datos.

• **Migración de Aplicaciones Legacy:** Ideal para software antiguo que requiere comunicarse en subredes L2 específicas o que rechaza NAT.

• **Requisito Modo Promiscuo:** La tarjeta física del host debe soportar modo promiscuo para aceptar múltiples direcciones MAC.

Complementos de Red y CNI

• **Ecosistema Extensible:** Docker permite instalar plugins de terceros vía `docker plugin install`.

• **Drivers CNI (Container Network Interface):** Estándar agnóstico de la CNCF para configurar interfaces de red en contenedores.

• **Calico / Cilium:** Plugins avanzados que incorporan políticas de seguridad por software (Network Policies), cifrado WireGuard y enrutamiento BGP.

• **Weave Net / Flannel:** Redes overlay de configuración rápida para entornos distribuidos.

Laboratorio 2: Redes Personalizadas y DNS Embebido

Objetivos y Pasos del Ejercicio Práctico

- **Paso 1:** Inspeccionar la red bridge por defecto (`docker network inspect bridge`) y comprobar que no resuelve nombres.
- **Paso 2:** Crear una red personalizada llamada `mi-red-app` (`docker network create --driver bridge mi-red-app`).
- **Paso 3:** Levantar un contenedor de base de datos Redis en `mi-red-app` (`docker run -d --name cache-redis --network mi-red-app redis:alpine`).
- **Paso 4:** Levantar un contenedor cliente en la misma red y comprobar la resolución de nombres DNS (`docker run --rm --network mi-red-app alpine ping -c 2 cache-redis`).
- **Paso 5:** Crear una segunda red `red-aislada` y verificar que los contenedores no pueden comunicarse entre redes distintas.
- **Paso 6:** Conectar en caliente un contenedor a dos redes mediante `docker network connect` y verificar la doble conectividad.
- **Paso 7:** Limpieza de redes y contenedores (`docker rm -f`, `docker network rm`).

MÓDULO 3: PERSISTENCIA Y VOLÚMENES

Mecanismos de Persistencia: Storage Drivers (Overlay2), Bind Mounts, Named Volumes y Tmpfs
Segunda mitad del Día 2 (2.5 Horas)

El Reto de los Datos: Efímero vs Persistente

La Naturaleza Efímera del Contenedor

• **Contenedores Desechables:** Los contenedores deben poder pararse, destruirse y recrearse sin temor a perder información valiosa.

• **La Capa R/W no es para Datos Críticos:** La capa de escritura del contenedor añade sobrecarga de rendimiento por el mecanismo Copy-on-Write.

• **Riesgo de Pérdida Accidental:** Ejecutar `docker rm -f` elimina la capa R/W para siempre sin opción a recuperación.

• **Dificultad de Backup:** Es complejo extraer datos masivos desde las capas internas de un storage driver en producción.

La Solución: Datos Fuera del Ciclo de Vida

• **Desacoplamiento Almacenamiento-Cómputo:** Los datos deben residir en volúmenes independientes del contenedor.

• **Rendimiento de E/S Nativo:** Los volúmenes omiten el Storage Driver y escriben directamente a la velocidad nativa del disco del host.

• **Compartición de Datos:** Varios contenedores pueden montar el mismo volumen de forma simultánea (ej. un contenedor escribe logs y otro los procesa).

• **Persistencia Garantizada:** Si el contenedor muere, los datos permanecen intactos en el almacenamiento.

Overlay2 a Fondo: La Magia del Union File System

Los 4 Directorios Clave de Overlay2

- **`lowerdir`:** Colección de directorios de solo lectura correspondientes a cada una de las capas de la imagen.
- **`upperdir`:** Directorio de lectura/escritura propio y exclusivo del contenedor activo.
- **`workdir`:** Directorio interno que utiliza Linux para preparar archivos antes de hacer la operación atómica de copia hacia el upperdir.
- **`merged`:** El punto de montaje unificado que ve el proceso del contenedor en `/`. Superpone el `upperdir` sobre los `lowerdir`.

$ Estructura de Grafos en Disco

```
# Inspeccionar los directorios de Overlay2 en un contenedor
$ docker inspect -f '{{json .GraphDriver.Data}}' mi-contenedor
{
  "LowerDir": "/var/lib/docker/overlay2/abc.../diff:...",
  "MergedDir": "/var/lib/docker/overlay2/def.../merged",
  "UpperDir": "/var/lib/docker/overlay2/def.../diff",
  "WorkDir": "/var/lib/docker/overlay2/def.../work"
}

# Cómo funciona la eliminación de un archivo:
# Si el contenedor borra un archivo de la imagen base,
# Overlay2 crea un archivo especial 'whiteout' (cdev 0,0)
# en el upperdir que oculta el archivo del lowerdir.
```

Mecanismos de Persistencia: Volumes, Binds y Tmpfs

Named Volumes

• **Ubicación:** `/var/lib/docker/volumes/`.

• **Gestión:** Completamente administrado por Docker.

• **Seguridad:** Aislado del acceso directo de usuarios del host.

• **Drivers:** Soporta plugins de cloud y almacenamiento en red.

• **Recomendado:** Bases de datos y almacenamiento en producción.

Bind Mounts

• **Ubicación:** Cualquier ruta del host (ej. `/home/user/app`).

• **Gestión:** Dependiente del sistema de archivos del host.

• **Acceso:** El host y el contenedor leen y escriben a la vez.

• **Riesgos:** Dependencia de rutas fijas y permisos de UID/GID.

• **Recomendado:** Desarrollo local con recarga en caliente (hot-reload).

Tmpfs Mounts

• **Ubicación:** Memoria RAM del host.

• **Gestión:** No escribe jamás en el disco físico.

• **Velocidad:** Rendimiento ultrarrápido en lectura/escritura.

• **Volatilidad:** Se destruye por completo al parar el contenedor.

• **Recomendado:** Datos altamente sensibles, tokens y cachés volátiles.

Volúmenes con Nombre (Named Volumes): Estándar de Producción

Ciclo de Vida de los Volúmenes

- **Independencia Total:** Los volúmenes tienen su propio ciclo de vida; no se borran al eliminar el contenedor que los utilizaba.
- **Inicialización Automática:** Si montas un volumen vacío sobre un directorio del contenedor que ya tiene archivos (ej. `/var/lib/mysql`), Docker copia automáticamente esos archivos al volumen.
- **Sintaxis Moderna `--mount`:** Recomendada frente al flag clásico `-v` por ser explícita y menos propensa a errores.
- **Limpieza de Volúmenes Huérfanos:** `docker volume prune` elimina aquellos volúmenes que no están siendo usados por ningún contenedor.

$ Demostración de Persistencia

```
# 1. Crear volumen explícito para PostgreSQL
$ docker volume create pgdata

# 2. Arrancar PostgreSQL montando el volumen
$ docker run -d --name db-postgres \
    --mount
source=pgdata,target=/var/lib/postgresql/data \
    -e POSTGRES_PASSWORD=mi_pass postgres:16-alpine

# 3. Destruir el contenedor simulando un desastre
$ docker rm -f db-postgres

# 4. Recrear el contenedor con una versión más
moderna
$ docker run -d --name db-postgres-nuevo \
    --mount
source=pgdata,target=/var/lib/postgresql/data \
    postgres:16-alpine

# ¡Todos los registros y tablas siguen intactos!
```

Bind Mounts: Desarrollo Ágil y Riesgos de Permisos

La Herramienta del Desarrollador

• **Edición en Vivo (Hot-Reloading):** Permite editar el código fuente en VS Code en el host y ver los cambios reflejados al instante dentro del contenedor sin reconstruir la imagen.

• **Sintaxis con Ruta Absoluta:** Requiere indicar la ruta completa del host (ej. `$(pwd):/app` o `type=bind,source=$(pwd),target=/app`).

• **Sobrescritura de Directorios:** CUIDADO: Montar una carpeta vacía del host sobre una carpeta con archivos del contenedor ocultará los archivos del contenedor.

• **Modo Solo Lectura (`ro`):** Excelente práctica de seguridad para archivos de configuración sensibles (`:ro` o `readonly`).

$ Uso Seguro de Bind Mounts

```
# Montar código local para desarrollo con recarga
$ docker run -d --name web-dev \
    -p 3000:3000 \
    -v "$(pwd)/src:/app/src:ro" \
    node:20-alpine npm run dev

# El Gran Problema de Permisos en Linux:
# Si el contenedor corre como root (UID 0), creará
# archivos en tu host propiedad de root.
# Solución: Pasar el UID del usuario actual:
$ docker run --user "$(id -u):$(id -g)" ...

# Sintaxis moderna explícita con --mount:
$ docker run --mount
type=bind,source=/etc/nginx/nginx.conf,\
target=/etc/nginx/nginx.conf,readonly nginx
```

Tmpfs Mounts: Almacenamiento en Memoria RAM

Almacenamiento Volátil de Alto Rendimiento

- **Directo a la Memoria RAM:** Los archivos se escriben exclusivamente en la memoria del host; jamás tocan los discos SSD/NVMe.
- **Cero Rastro en Disco:** Al detener el contenedor, los datos desaparecen instantáneamente sin dejar fragmentos recuperables en el sistema de archivos.
- **Protección de Secretos en Memoria:** Ideal para almacenar certificados temporales, tokens OAuth, llaves criptográficas privadas o credenciales de sesión.
- **Límites de Tamaño (`tmpfs-size`):** Es fundamental limitar el tamaño de tmpfs para evitar que un contenedor llene la RAM del host con archivos temporales.

$ Montaje Tmpfs en Terminal

```
# Montar /tmp en memoria RAM con límite de 64MB
$ docker run -d --name app-segura \
    --tmpfs
/app/secrets:rw,noexec,nosuid,size=64m \
    nginx

# Comprobar que está montado como tmpfs
$ docker exec app-segura df -h /app/secrets
Filesystem  Size  Used  Avail  Use%  Mounted on
tmpfs       64M     0    64M    0%  /app/secrets

# Sintaxis con --mount:
$ docker run --mount
type=tmpfs,destination=/app/cache,\
    tmpfs-size=128m,tmpfs-mode=1777 mi-app
```

Contenedores de Volúmenes de Datos: Perspectiva Histórica

El Patrón '--volumes-from' (Histórico)

• **El Problema en 2014:** En las primeras versiones de Docker no existía el comando `docker volume create` ni existían Named Volumes.

• **La Solución Ingeniosa:** Crear un contenedor 'tonto' (Data Volume Container) que definía volúmenes y permanecía detenido (`Exited`).

• **Consumo con `--volumes-from`:** Otros contenedores montaban los volúmenes de ese contenedor detenido usando el flag `--volumes-from <nombre>`.

• **Persistencia Compartida:** Varios contenedores podían leer y escribir en los mismos datos mientras el contenedor base existiera.

Por Qué Hoy Está en Desuso

• **Llegada de Docker 1.9 (2015):** Se introdujo el subsistema completo de gestión de volúmenes (`docker volume`).

• **Mayor Claridad y Desacoplamiento:** Los volúmenes son ciudadanos de primera clase con nombre propio, sin necesidad de mantener contenedores fantasmas parados.

• **Plugins de Almacenamiento Modernos:** Los Named Volumes admiten drivers externos (NFS, AWS EBS, Azure Disk), algo inviable con el patrón antiguo.

• **Conclusión del Temario:** Se estudia para comprender código legacy o configuraciones antiguas, pero hoy se DEBE utilizar `docker volume`.

Drivers de Almacenamiento Remoto y Red

Almacenamiento Distribuido (Cloud y NAS)

• **El Límite del Almacenamiento Local:** Un Named Volume local está atado al disco del servidor físico donde corre el daemon.

• **Drivers de Volumen (`-d <driver>`):** Permiten que Docker monte sistemas de archivos remotos directamente en el contenedor.

• **Almacenamiento NFS / CIFS:** Conectar carpetas compartidas de un NAS / SAN corporativo mediante el driver `local` nativo con opciones de red.

• **Almacenamiento en Cloud:** Plugins para montar volúmenes elásticos de nube (ej. AWS EBS, Azure Files, Google Cloud Persistent Disk).

$ Montaje de Almacenamiento NFS

```
# Ejemplo: Crear un volumen Docker respaldado por NFS
$ docker volume create --driver local \
    --opt type=nfs \
    --opt o=addr=192.168.1.100,rw,noatime,rsize=8192 \
    --opt device=:/srv/nfs/datos_compartidos \
    volumen-nfs

# Montar el volumen en cualquier contenedor
$ docker run -d --name app-nfs \
    -v volumen-nfs:/app/storage \
    mi-aplicacion

# Si el contenedor se traslada a otro nodo del clúster,
# montará el mismo volumen NFS con los mismos datos.
```

Estrategias de Backup y Migración de Volúmenes

El Patrón Contenedor de Backup

• **Sin Parar el Motor:** No es necesario acceder físicamente como root a `/var/lib/docker/volumes` ni detener el daemon.

• **Contenedor Efímero de Backup:** Se arranca un contenedor ligero (ej. `alpine`) que monta simultáneamente el Named Volume a respaldar y una carpeta del host donde guardar el `.tar.gz`.

• **Automatización en Cron / CI:** El proceso se empaqueta en una única línea de comando reproducible para tareas programadas de madrugada.

• **Restauración Limpia:** El proceso inverso descomprime el `.tar.gz` dentro de un volumen nuevo o existente.

$ Comandos de Backup y Restore

```
# 1. BACKUP: Empaquetar el volumen 'pgdata' en
un .tar.gz
$ docker run --rm \
    -v pgdata:/volumen_origen:ro \
    -v $(pwd)/backups:/backup_destino \
    alpine tar czf /backup_destino/pgdata_$(date +
%F).tar.gz \
          -C /volumen_origen .

# 2. RESTAURAR: Descomprimir en un nuevo volumen
$ docker volume create pgdata_recuperado
$ docker run --rm \
    -v pgdata_recuperado:/volumen_destino \
    -v $(pwd)/backups:/backup_destino \
    alpine tar xzf /backup_destino/pgdata_2026-10-
27.tar.gz \
          -C /volumen_destino
```

Buenas Prácticas en Gestión de Persistencia

Reglas de Oro en Producción

• **Jamás Guardar Datos en Capas R/W:** Todo dato que deba sobrevivir al contenedor debe residir en un Named Volume o Bind Mount.

• **Nombres Significativos para Volúmenes:** Evitar volúmenes anónimos huérfanos generados accidentalmente con flags `-v /ruta`.

• **Separar Discos para Docker:** En servidores Linux de producción, montar `/var/lib/docker` en una partición de disco o volumen LVM separado para que un contenedor desbocado no llene el root `/` del host.

• **Etiquetado de Volúmenes:** Usar labels en volúmenes (`--label env=prod`) para automatizar auditorías y políticas de retención.

Antipatrones a Erradicar

• **Antipatrón 'docker commit':** Modificar un contenedor a mano y hacer `docker commit` para 'guardar los cambios' es el enemigo de la reproducibilidad.

• **Bind Mounts con Rutas Absolutas de Portátil:** Subir a producción Dockerfiles o Compose con rutas de tipo `/Users/juan/proyecto` o `C:\Users\...`.

• **Ignorar el Permiso de Escritura:** Olvidar comprobar si el usuario sin privilegios del contenedor (`USER app`) tiene permisos de escritura en la carpeta montada.

• **Acumulación de Volúmenes Muertos:** No programar limpiezas periódicas (`docker volume prune`), provocando el llenado silencioso de discos.

Laboratorio 3: Persistencia de BD y Hot-Reloading

Objetivos y Pasos del Ejercicio Práctico

- **Parte A (Named Volume):** Crear un Named Volume `db_datos` e iniciar una base de datos MariaDB/PostgreSQL.
- **Paso 2:** Conectarse a la base de datos vía CLI (`docker exec -it ...`), crear una tabla e insertar registros de prueba.
- **Paso 3:** Destruir intencionadamente el contenedor de base de datos (`docker rm -f`).
- **Paso 4:** Arrancar un contenedor nuevo con la misma versión o superior montando `db_datos` y verificar que los registros siguen intactos.
- **Parte B (Bind Mount):** Crear una aplicación web estática o API Node/Python en el host.
- **Paso 6:** Arrancar el contenedor montando el código con Bind Mount (`-v $(pwd)/src:/app:ro`).
- **Paso 7:** Modificar un archivo HTML/JS en el editor del host y comprobar con `curl` que el cambio se refleja inmediatamente en el navegador sin reconstruir la imagen.

RESUMEN DE LA JORNADA 2

Conceptos consolidados: Drivers de red (Bridge, Host, None, Overlay, Macvlan), DNS interno, Overlay2, Named Volumes, Bind Mounts y Tmpfs.
Próxima Jornada: Módulo 4 (Contenedores en Cloud: Azure, AWS, GCP y Registries).

CONTENEDORES EN CLOUD

Módulo 4: Containers en Azure, AWS, Google Cloud y Evolución de Docker Cloud
Jornada 3 de 4 (5 Horas Lectivas)

Evolución Cloud: De IaaS a CaaS y Serverless Containers

IaaS (VMs Tradicionales)

• **Gestión de Infraestructura:** Parchear el SO, gestionar kernels, configurar firewalls de red y agentes de seguridad.

• **Sobrecarga de Operaciones:** Escalar levantando nuevas VMs tarda minutos.

• **Pago por Capacidad:** Facturación continua 24/7 aunque la CPU esté al 1% de uso.

• **Uso:** Monolitos antiguos o cargas que requieren control extremo de hardware.

CaaS (Containers as a Service)

• **Control del Contenedor:** El usuario entrega la imagen Docker; el proveedor gestiona el runtime.

• **Orquestación Integrada:** Servicios que manejan el ciclo de vida, balanceo y reinicio de contenedores.

• **Escalado Ágil:** Adición de réplicas en segundos en lugar de minutos.

• **Ejemplos:** Amazon ECS, Azure Container Apps, Google Cloud Run.

Serverless Containers

• **Cero Servidores que Administrar:** Sin nodos, sin clusters que actualizar, sin capacidad ociosa.

• **Escalado a Cero:** Si no hay peticiones, el servicio escala a 0 réplicas y el coste es exactamente 0€.

• **Pago por Consumo Exacto:** Facturación al milisegundo por vCPU y GB consumidos.

• **Modelo Predilecto:** APIs REST, microservicios orientados a eventos y jobs batch.

El Ciclo de Vida Cloud: Build, Ship & Run

El Flujo Canónico del Contenedor

- **1. BUILD (Local o CI/CD):** El desarrollador o el servidor de integración continua (GitHub Actions, GitLab CI, Azure DevOps) compila la imagen.
- **2. TAG:** Se etiqueta la imagen con la URL del registro privado y el commit SHA o versión semántica (`mi-registry.azurecr.io/app:v1.2.0`).
- **3. SHIP (Push):** Se sube la imagen al Container Registry mediante protocolos HTTPS autenticados con tokens seguros.
- **4. RUN (Pull & Deploy):** El servicio cloud descarga la imagen desde el registry privado y arranca el contenedor.

$ Comandos de Flujo Cloud

```
# Flujo típico en terminal:

# 1. Autenticarse en el registro cloud
$ az acr login --name miregistry    # Azure
$ aws ecr get-login-password | ...  # AWS
$ gcloud auth configure-docker      # GCP

# 2. Etiquetar la imagen local con la ruta completa
$ docker tag mi-app:latest \
    miregistry.azurecr.io/servicios/mi-app:1.2.0

# 3. Enviar las capas al registro remoto
$ docker push miregistry.azurecr.io/servicios/mi-
app:1.2.0

# 4. Desplegar en servicio serverless
$ az container create ... --image
miregistry...:1.2.0
```

Container Registries: Seguridad, Escaneo y Gobernanza

Docker Hub vs Registries Cloud Gestionados

• **Docker Hub:** El registro público central de la comunidad. Excelente para imágenes base oficiales (Ubuntu, Alpine, Node, Python), pero con límites de rate-limit (peticiones anónimas restringidas).

• **Registries Privados Cloud (ACR, ECR, Artifact Registry):** Almacenamiento privado dentro de la red del proveedor de nube sin límites de ancho de banda y con coste mínimo.

• **Autenticación Nativa IAM:** Acceso basado en identidades gestionadas y roles de nube (evita contraseñas estáticas en archivos de configuración).

• **Geo-Replicación:** Duplicación automática de imágenes en múltiples regiones geográficas para descargas ultrarrápidas y resiliencia.

Seguridad y Escaneo de Vulnerabilidades

• **Escaneo Automático de CVEs:** Los registros analizan los paquetes del SO de la imagen (apt, apk, rpm) y alertan sobre vulnerabilidades conocidas (CVE-xxxx).

• **Firma Criptográfica (Notary / Cosign):** Garantía de que la imagen desplegada en producción fue firmada por el equipo de seguridad y no ha sido alterada.

• **Políticas de Inmutabilidad de Tags:** Impedir que un tag existente (ej. `:v1.0.0`) sea sobrescrito por un push malicioso o accidental.

• **Políticas de Limpieza Automática (Lifecycle Rules):** Eliminación programada de imágenes sin etiquetar (untagged) tras 30 días para ahorrar costes.

Contenedores en Microsoft Azure: Panorámica de Servicios

El Catálogo de Soluciones de Contenedores de Azure

• **Azure Container Registry (ACR):** Servicio gestionado de registro privado compatible con OCI, con cifrado en reposo, soporte de geo-replicación y control de acceso RBAC vía Microsoft Entra ID (Azure AD).

• **Azure Container Instances (ACI):** La forma más rápida y simple de ejecutar un contenedor en Azure sin aprovisionar máquinas virtuales ni clústeres. Ideal para jobs puntuales, tareas batch o despliegues simples.

• **Azure Container Apps (ACA):** Plataforma de aplicaciones serverless basada en Kubernetes gestionado, Envoy y KEDA (Kubernetes Event-driven Autoscaling). Pensada para microservicios con escalado automático.

• **Azure App Service for Containers:** Ejecución de aplicaciones web contenerizadas con certificados SSL automáticos, dominios personalizados, slots de despliegue y autenticación integrada.

• **Azure Kubernetes Service (AKS):** Servicio completo y gestionado de Kubernetes para despliegues empresariales de gran escala con control fino de infraestructura.

Azure Container Instances (ACI): Contenedores sin Servidores

Características Principales de ACI

- **Arranque Inmediato:** Puesta en marcha en segundos sin esperas de aprovisionamiento de máquinas virtuales.
- **Facturación por Segundos:** Pago únicamente por la duración exacta de ejecución y los recursos asignados (vCPUs y GB de memoria).
- **Aislamiento por Hipervisor:** Seguridad multinquilino de nivel hipervisor; cada instancia corre en su propio sandbox de virtualización ligera.
- **Dirección IP Pública y DNS:** Asignación automática de IP pública y nombre de dominio FQDN (`<nombre>.eastus.azurecontainer.io`).
- **Montaje de Volúmenes:** Soporte nativo para montar carpetas compartidas de Azure Files mediante SMB/CIFS.

$ Despliegue con Azure CLI

```
# Desplegar un contenedor Nginx en Azure con Azure CLI
$ az container create \
    --resource-group rg-formacion \
    --name mi-web-aci \
    --image mcr.microsoft.com/azuredocs/aci-helloworld \
    --dns-name-label curso-docker-demo \
    --ports 80 \
    --cpu 1 \
    --memory 1.5

# Comprobar estado y ver FQDN asignado
$ az container show --resource-group rg-formacion \
    --name mi-web-aci --query "ipAddress.fqdn"
"curso-docker-demo.eastus.azurecontainer.io"

# Destruir la instancia al finalizar
$ az container delete --resource-group rg-formacion \
    --name mi-web-aci -y
```

Azure Container Apps (ACA): Microservicios y KEDA

La Plataforma Moderna de Microservicios

• **Kubernetes Invisible:** Desarrollado sobre clústeres de AKS administrados por Microsoft; disfrutas de los beneficios de K8s sin gestionar manifests complejos.

• **Escalado Orientado a Eventos (KEDA):** Escala no solo por CPU/RAM, sino por número de peticiones HTTP concurrentes, mensajes en cola (RabbitMQ, Azure Service Bus) o eventos Kafka.

• **Escalado a Cero:** Apaga completamente los contenedores cuando no hay tráfico activo, minimizando costes.

• **Descubrimiento de Servicios Integrado:** Comunicación interna por DNS privado entre microservicios del mismo entorno.

• **Soporte Nativo de Dapr:** Integración opcional con Distributed Application Runtime para patrones de microservicios (pub/sub, state store).

Casos de Uso Ideales para ACA

• **APIs Públicas y Privadas:** Microservicios HTTP con balanceador de carga Envoy y certificados HTTPS automáticos.

• **Procesadores de Colas en Segundo Plano:** Workers que se activan solo cuando entran mensajes a procesar y se apagan al terminar.

• **Despliegues Azules/Verdes (Blue/Green):** Capacidad de dividir tráfico porcentual entre dos versiones distintas de la misma aplicación.

• **Alternativa Práctica a Kubernetes:** Para equipos que necesitan orquestación elástica sin la sobrecarga operativa de administrar un clúster K8s.

Contenedores en Amazon Web Services (AWS)

Los Cuatro Pilares de Contenedores en AWS

• **Amazon ECR (Elastic Container Registry):** Registro de contenedores totalmente administrado, cifrado con KMS, con escaneo de seguridad y replicación entre regiones.

• **Amazon ECS (Elastic Container Service):** El orquestador nativo de AWS, altamente optimizado, maduro y profundamente integrado con servicios como IAM, CloudWatch, Application Load Balancer y Route 53.

• **AWS Fargate:** Motor de cómputo serverless para Amazon ECS y Amazon EKS. Permite ejecutar contenedores sin tener que gestionar servidores EC2 subyacentes.

• **Amazon EKS (Elastic Kubernetes Service):** Servicio de Kubernetes certificado por la CNCF, altamente escalable y estándar en la industria para arquitecturas multi-cloud.

• **AWS App Runner:** Servicio serverless de despliegue directo desde código o contenedor para aplicaciones web y APIs con autoescalado automático.

Amazon ECS: EC2 Launch Type vs AWS Fargate

Modelo EC2 Launch Type

• **Control Total de Servidores:** Los contenedores se ejecutan sobre instancias EC2 que tú aprovisionas y administras.

• **Responsabilidad Operativa:** Debes parchear el SO de las instancias, actualizar el agente de ECS y monitorizar el espacio en disco.

• **Aprovechamiento de Descuentos:** Permite usar instancias reservadas, Savings Plans o instancias Spot (hasta un 90% de descuento para cargas interrumpibles).

• **Tipos Especiales de Hardware:** Obligatorio si necesitas GPUs dedicadas (entrenamiento IA), memoria extrema o tipos de CPU específicos.

Modelo AWS Fargate (Serverless)

• **Cero Instancias EC2:** Defines tu tarea de contenedor con vCPU y memoria; AWS aprovisiona el entorno al vuelo.

• **Aislamiento Estricto por Tarea:** Cada tarea de Fargate corre en su propia máquina virtual aislada (tecnología Firecracker MicroVMs).

• **Mantenimiento Cero:** AWS gestiona los parches de seguridad del host, el kernel y el daemon de contenedores.

• **Modelo Recomendado por AWS:** Es la opción predeterminada para el 90% de las cargas de trabajo por su drástica reducción de horas de ingeniería de operaciones.

Amazon ECR: Seguridad y Políticas de Ciclo de Vida

Funcionalidades Clave de ECR

- **Autenticación Efímera IAM:** Las contraseñas de Docker obtenidas con `get-login-password` caducan automáticamente tras 12 horas.
- **Políticas de Repositorio (Resource Policies):** Control granular de qué cuentas de AWS o roles IAM pueden hacer push o pull.
- **Escaneo Continuo con Amazon Inspector:** Detección automática en tiempo real de vulnerabilidades en paquetes del SO y dependencias de lenguaje (Python, Java, JS).
- **Inmutabilidad de Tags:** Protección contra sobrescritura accidental de imágenes de producción mediante configuración declarativa.

$ Uso de Amazon ECR

```
# 1. Obtener token de login de ECR y pasarlo a Docker
$ aws ecr get-login-password --region eu-west-1 | \
    docker login --username AWS --password-stdin \
    123456789012.dkr.ecr.eu-west-1.amazonaws.com

# 2. Crear repositorio privado
$ aws ecr create-repository --repository-name app-backend \
    --image-tag-mutability IMMUTABLE \
    --image-scanning-configuration scanOnPush=true

# 3. Subir la imagen
$ docker push 123456789012.dkr.../app-backend:1.0
```

Contenedores en Google Cloud Platform (GCP)

La Cuna de Kubernetes y la Contenedorización Masiva

• **Pioneros en Contenedores:** Google lleva más de 15 años ejecutando toda su infraestructura (Gmail, Search, YouTube) en contenedores mediante su sistema interno 'Borg' (el predecesor directo de Kubernetes).

• **Google Artifact Registry:** La evolución moderna de Google Container Registry (GCR). Soporta imágenes OCI, paquetes de software (npm, pip, maven) y escaneo de vulnerabilidades.

• **Google Cloud Run:** Plataforma serverless totalmente administrada que ejecuta contenedores invocables mediante peticiones HTTP o mensajes de Eventarc/PubSub.

• **Google Kubernetes Engine (GKE):** El servicio de Kubernetes administrado de referencia en el mercado, con soporte para GKE Autopilot (gestión serverless del plano de datos).

• **Aislamiento gVisor:** Tecnología de sandbox de kernel desarrollada por Google que intercepta syscalls para máxima seguridad multinquilino.

Google Cloud Run: La Excelencia en Serverless Containers

Por Qué Cloud Run Cambió las Reglas

• **Cualquier Lenguaje o Binario:** A diferencia de AWS Lambda (restringido a ciertos runtimes), en Cloud Run puedes ejecutar cualquier código empaquetado en un contenedor Docker que escuche en `$PORT`.

• **Concurrencia por Contenedor:** Una única instancia puede atender hasta 1000 peticiones simultáneas, abaratando costes drásticamente frente a funciones serverless tradicionales.

• **Escalado Instantáneo de 0 a 1000+:** Reacciona en milisegundos a avalanchas repentinas de tráfico web.

• **Dominio HTTPS y CDN Automático:** Cada servicio recibe un subdominio `.run.app` con TLS gestionado y balanceo global Anycast de Google.

• **Pago Exacto por Petición:** Si no hay tráfico, el coste es cero.

$ Despliegue con gcloud CLI

```
# Desplegar un contenedor en Cloud Run con un solo comando
$ gcloud run deploy mi-api-rest \
    --image gcr.io/cloudrun/hello \
    --platform managed \
    --region europe-west1 \
    --allow-unauthenticated \
    --port 8080 \
    --memory 512Mi \
    --min-instances 0 \
    --max-instances 10

# URL lista para producción generada en segundos:
# Service URL: https://mi-api-rest-xyz-ew.a.run.app
```

Google Artifact Registry: Almacén Universal OCI

La Evolución de Container Registry (GCR)

- **Soporte Multipaquete Unificado:** Un único servicio para almacenar imágenes Docker OCI, paquetes npm, paquetes Python (PyPI), Java Maven y charts de Helm.
- **Repositorios Regionales:** Ubicación precisa de los artefactos en la misma región que tus servicios de Cloud Run o GKE para reducir la latencia de pull a milisegundos.
- **Control de Acceso IAM:** Integración granular con Cuentas de Servicio (Service Accounts) de Google Cloud.
- **Vulnerability Scanning y Binary Authorization:** Bloqueo automático del despliegue en GKE si la imagen no supera las políticas de seguridad corporativas.

$ Flujo Artifact Registry

```
# Configurar Docker para autenticarse con Artifact Registry
$ gcloud auth configure-docker europe-west1-docker.pkg.dev

# Etiquetar imagen con formato Artifact Registry
$ docker tag mi-app:1.0 \
    europe-west1-docker.pkg.dev/mi-proyecto/repo-apps/mi-app:1.0

# Enviar la imagen
$ docker push \
    europe-west1-docker.pkg.dev/mi-proyecto/repo-apps/mi-app:1.0
```

Docker Cloud: Análisis Histórico y su Evolución

Qué fue 'Docker Cloud' (2015-2018)

• **Origen:** Nació de la adquisición de la startup Tutum por parte de Docker Inc. en octubre de 2015.

• **Propósito Original:** Proporcionaba un panel web SaaS para gestionar, desplegar y escalar contenedores Docker conectando credenciales de AWS, Azure y DigitalOcean.

• **Cierre Oficial:** Docker Inc. anunció su retirada definitiva el 21 de mayo de 2018 tras el auge arrollador de Kubernetes y los servicios administrados de los gigantes cloud.

• **¿Por qué aparece en el temario?:** Es un vestigio histórico de programas formativos clásicos que todo instructor profesional debe contextualizar con rigor.

Hacia Dónde Evolucionó Docker Inc.

• **Docker Hub Moderno:** El registro público de referencia, con soporte de Builds automatizados y organizaciones.

• **Docker Desktop & Extensions:** Plataforma de productividad para desarrolladores (Mac, Windows, Linux) con extensiones de Kubernetes, Snyk, Portainer.

• **Docker Scout:** Herramienta moderna de análisis de la cadena de suministro de software, SBOM (Software Bill of Materials) y recomendaciones de remediación de CVEs.

• **Enfoque Actual:** Docker se enfoca en la 'experiencia del desarrollador' (DevX) mientras delega la infraestructura cloud a AWS, Azure y GCP.

¿Cuándo usar CaaS Serverless vs Kubernetes?

Elegir CaaS Serverless (Cloud Run, ACI, Fargate)

• **Equipos Pequeños / Startups:** Cero personas dedicadas a administrar infraestructura, redes o parches de clúster.

• **Cargas Web y APIs HTTP:** Cargas de trabajo orientadas a peticiones que se benefician del escalado automático a cero.

• **Procesamiento Batch / Eventos:** Tareas que arrancan, procesan datos durante unos minutos y se apagan.

• **Menor Tiempo al Mercado:** Poner en producción una imagen en 5 minutos con un único comando de CLI.

• **Presupuesto Reducido:** No pagas por un plano de control fijo ni por nodos ociosos.

Elegir Kubernetes Gestionado (AKS, EKS, GKE)

• **Topologías Complejas:** Cientos de microservicios con dependencias complejas, mallas de servicios (Service Mesh como Istio/Linkerd).

• **Control Fino de Red y Almacenamiento:** Necesidad de definir políticas de red L3/L4 personalizadas, volúmenes de bloque y topologías multi-nodo avanzadas.

• **Ecosistema CNCF:** Uso de operadores de bases de datos, herramientas GitOps (ArgoCD, Flux) y observabilidad masiva.

• **Multi-Cloud / Híbrido:** Evitar vendor lock-in ejecutando la misma especificación de Kubernetes en cualquier nube y on-premise.

• **Equipo Dedicado de Platform / DevOps:** Capacidad para asumir la gestión del clúster y sus actualizaciones periódicas.

Buenas Prácticas para Contenedores en Producción Cloud

Diseño y Operación Resiliente

• **Diseño 'Stateless' (Sin Estado):** No guardar sesiones ni archivos en disco local; delegar el estado a bases de datos cloud (RDS, Cloud SQL, CosmosDB) y objetos (S3, GCS, Blob Storage).

• **Configuración por Variables de Entorno:** Cumplir el factor III de 'The Twelve-Factor App'. Inyectar configuración y secretos mediante los gestores de secretos de la nube (Secret Manager, Key Vault).

• **Health Checks Nativos (`HEALTHCHECK`):** Configurar sondas de liveness y readiness para que el orquestador cloud reinicie instancias que no responden.

• **Logs en Salida Estándar (stdout/stderr):** Escribir siempre los logs en stdout; el agente cloud los recolectará automáticamente hacia CloudWatch, Cloud Logging o Azure Monitor.

Seguridad y Optimización

• **Prohibido Hardcodear Secretos en la Imagen:** Jamás escribir API keys o contraseñas en el Dockerfile o durante el build.

• **Usuario No Root Obligatorio:** Configurar directiva `USER appuser` para impedir escaladas de privilegios en el host cloud.

• **Fijar Límites de Recursos:** Declarar siempre CPU y memoria máximos en la definición de la tarea o servicio cloud.

• **Monitorización de Costes y Autoescalado:** Fijar un número máximo de instancias (`max-instances`) para evitar sorpresas desagradables en la factura mensual ante un ataque DoS.

Laboratorio 4: Registries Cloud y Despliegue CaaS

Objetivos y Pasos del Ejercicio Práctico

• **Paso 1:** Configurar una cuenta en Docker Hub o registry privado del proveedor cloud (Azure ACR / AWS ECR / Google Artifact Registry).

• **Paso 2:** Autenticar el cliente local de Docker mediante terminal segura (`docker login` o credenciales CLI).

• **Paso 3:** Construir una versión 1.0.0 de nuestra API con tag semántico siguiendo la convención: `<registry>/<namespace>/<app>:<version>`.

• **Paso 4:** Ejecutar `docker push` y verificar en el panel web del registry la carga de capas y el hash SHA256 generado.

• **Paso 5:** Desplegar el contenedor en un servicio serverless (ej. Azure ACI, Google Cloud Run o simulación con entorno local) exponiendo el puerto público.

• **Paso 6:** Probar la URL pública asignada mediante navegador o `curl`.

• **Paso 7:** Realizar una modificación de código, subir la versión `1.0.1` y ejecutar una actualización sin caída de servicio (Zero Downtime).

RESUMEN DE LA JORNADA 3

Conceptos consolidados: CaaS, Serverless Containers, Azure (ACI, ACA, ACR), AWS (ECS, Fargate, ECR), Google Cloud (Cloud Run, Artifact Registry) y Registries OCI.
Próxima Jornada: Módulo 5 (Orquestación a escala, Kubernetes y Taller Final Integrador).

ORQUESTACIÓN Y KUBERNETES

Módulo 5: ¿Qué es Orquestación?, Comparativa de Orquestadores y Arquitectura de Kubernetes
Jornada 4 de 4 (5 Horas Lectivas)

¿Por qué Orquestar? Los Límites de Docker Standalone

Los Límites de un Solo Host

• **Punto Único de Fallo (SPOF):** Si el servidor donde corre el daemon de Docker cae por avería de hardware o corte de luz, todos los servicios caen.

• **Capacidad Limitada:** Un único servidor físico tiene un tope finito de CPU y memoria RAM; no puede escalar indefinidamente (límite del escalado vertical).

• **Sin Auto-Recuperación Multi-Servidor:** La directiva `--restart always` solo reinicia el contenedor si el host sigue vivo; no puede moverlo a otra máquina.

• **Gestión Manual Inviable:** Desplegar y monitorizar manualmente 100 contenedores con `docker run` o Docker Compose en múltiples IPs se vuelve ingobernable.

La Promesa del Orquestador

• **Abstracción del Clúster:** Unifica decenas de servidores en un único grupo de recursos lógicos de cómputo, red y almacenamiento.

• **Auto-Sanación (Self-Healing):** Si un nodo físico muere, el orquestador detecta la pérdida y reprograma automáticamente los contenedores en nodos sanos.

• **Balanceo de Carga y Enrutamiento:** Distribución inteligente de peticiones entrantes entre todas las réplicas disponibles.

• **Actualizaciones Progresivas (Rolling Updates):** Actualización a nuevas versiones de código sin un solo segundo de parada de servicio.

Las 5 Capacidades Clave de un Orquestador

Pilares Funcionales de la Orquestación Moderna

• **1. Scheduling Inteligente:** Evalúa requisitos de CPU, memoria, afinidad y tolerancia a fallos para decidir en qué servidor exacto debe ejecutarse cada contenedor.

• **2. Auto-Sanación (Self-Healing):** Monitoriza continuamente la salud de los procesos; si un contenedor falla en sus sondas de salud, lo destruye y levanta uno nuevo.

• **3. Escalado Horizontal Dinámico (HPA):** Aumenta o disminuye el número de réplicas del contenedor de forma automática en función de la métrica de CPU, memoria o tráfico.

• **4. Descubrimiento de Servicios y Balanceo de Carga:** Asigna una dirección IP virtual estable y un nombre DNS a cada conjunto de réplicas, balanceando el tráfico TCP/UDP.

• **5. Despliegues y Rollbacks Automatizados:** Despliega la versión nueva contenedor por contenedor; si detecta errores en las sondas, revierte el cambio al instante al estado anterior.

Comparativa de Orquestadores: Swarm vs K8s vs Nomad

Docker Swarm

• **Filosofía:** Simplicidad radical e integración nativa.

• **Curva de Aprendizaje:** Muy baja (reutiliza sintaxis de Docker Compose).

• **Configuración:** Se activa con `docker swarm init` en 10 segundos.

• **Ecosistema:** Limitado, desarrollo mantenido principalmente por Mirantis.

• **Ideal para:** Equipos pequeños y clústeres de tamaño modesto (1-15 nodos).

Kubernetes (K8s)

• **Filosofía:** Estándar industrial absoluto de la CNCF.

• **Curva de Aprendizaje:** Alta / Compleja (gran variedad de conceptos y APIs).

• **Configuración:** Complejo on-premise; simple en Cloud gestionado (EKS, AKS, GKE).

• **Ecosistema:** El mayor del mundo del software (Helm, ArgoCD, Istio, Prometheus).

• **Ideal para:** Cualquier empresa con arquitecturas de microservicios a escala.

HashiCorp Nomad

• **Filosofía:** Orquestador flexible, ligero y declarativo.

• **Flexibilidad de Cargas:** No solo corre contenedores; corre binarios Java, ejecutables y VMs.

• **Simplicidad Operativa:** Un solo binario Go tanto para servidor como para cliente.

• **Integración HashiCorp:** Nativo con Consul (red/DNS) y Vault (secretos).

• **Ideal para:** Entornos híbridos que mezclan contenedores y aplicaciones legacy.

Docker Swarm: El Orquestador Integrado

Arquitectura de Docker Swarm

- **Nativo en Docker Engine:** No requiere instalar nada adicional; viene apagado por defecto dentro del daemon `dockerd`.
- **Nodos Manager y Worker:** Los Managers coordinan el clúster usando el algoritmo de consenso Raft (mínimo 3 o 5 para quórum); los Workers ejecutan tareas.
- **Servicios (`docker service`):** Reemplaza a `docker run` para definir réplicas deseadas (ej. `--replicas 5`).
- **Ingress Routing Mesh:** Los puertos publicados se exponen en TODOS los nodos del clúster; cualquier nodo redirige la conexión al contenedor adecuado.

$ Comandos Básicos de Docker Swarm

```
# 1. Inicializar el nodo actual como Manager
$ docker swarm init
Swarm initialized: current node is now a manager.

# 2. Crear un servicio replicado de Nginx con 3 réplicas
$ docker service create --name web-cluster \
    --replicas 3 \
    -p 80:80 \
    nginx:alpine

# 3. Escalar el servicio en caliente
$ docker service scale web-cluster=10

# 4. Realizar actualización continua sin parada
$ docker service update --image nginx:1.25 web-cluster

# 5. Salir del modo Swarm
$ docker swarm leave --force
```

Introducción a Kubernetes: El Estándar de la Industria

Historia y Misión de K8s

• **Etimología:** Del griego κυβερνήτης (*kubernētēs*), que significa 'timonel' o 'capitán de barco' (de ahí el logo del timón de 7 radios).

• **Abreviatura 'K8s':** Porque hay 8 letras entre la 'K' y la 's'.

• **Herencia de Google Borg:** Creado por ingenieros de Google en 2014 basándose en una década de experiencia gestionando Borg y Omega.

• **Donación a la CNCF:** Google donó el proyecto a la Cloud Native Computing Foundation (Linux Foundation) para garantizar gobernanza neutra.

• **El Sistema Operativo de la Nube:** Hoy es el sustrato universal soportado por todos los grandes proveedores tecnológicos del planeta.

Los Principios de Diseño de K8s

• **Enfoque Declarativo:** Tú describes el **QUÉ** quieres (en archivos YAML); Kubernetes averigua el **CÓMO** conseguirlo y mantenerlo.

• **Inmutabilidad de la Infraestructura:** No se modifican contenedores en caliente; se reemplazan por nuevas instancias.

• **Desacoplamiento Absoluto:** Los componentes se comunican exclusivamente a través de la API REST central (`kube-apiserver`).

• **Extensibilidad Extrema:** Puedes ampliar Kubernetes con tus propios tipos de objetos mediante CRDs (Custom Resource Definitions) y Operadores.

Arquitectura de Kubernetes: Control Plane y Workers

CONTROL PLANE (Cerebro del Clúster)

- kube-apiserver: Puerta de entrada REST API y validación
- etcd: Almacén clave-valor distribuido del estado
- kube-scheduler: Asigna Pods a Nodos según recursos
- kube-controller-manager: Bucles de reconciliación

WORKER NODE (Ejecución de Cargas)

- kubelet: Agente de nodo que reporta al API Server
- kube-proxy: Gestión de reglas de red y Service IPs
- Container Runtime: containerd / CRI-O
- Pods: Contenedores de la aplicación (Carga real)

El Plano de Control (Control Plane) al Detalle

Componentes del Master

- **`kube-apiserver`:** El único punto de entrada de todo el clúster. Expone la API REST, autentica a los usuarios y valida los esquemas JSON/YAML.
- **`etcd`:** Base de datos distribuida clave-valor altamente consistente (protocolo Raft). Es la ÚNICA fuente de la verdad donde se almacena todo el estado del clúster.
- **`kube-scheduler`:** Observa los Pods recién creados que no tienen nodo asignado y elige el mejor nodo según recursos libres, afinidad y restricciones.
- **`kube-controller-manager`:** Ejecuta los bucles de control (Node Controller, Deployment Controller, Endpoint Controller) que fuerzan al clúster a alcanzar el estado deseado.

$ Interacción con el API Server

```
# ¿Cómo interactúa el administrador con el API Server?
# Utiliza la herramienta oficial de línea de comandos: kubectl

$ kubectl get nodes
NAME             STATUS   ROLES           AGE   VERSION
k8s-control-01   Ready    control-plane   30d   v1.29.2
k8s-worker-01    Ready    <none>          30d   v1.29.2
k8s-worker-02    Ready    <none>          30d   v1.29.2

# Todo comando de kubectl es traducido a llamadas
# HTTP REST hacia el kube-apiserver (puerto 6443):
# GET /api/v1/nodes
```

Los Nodos de Trabajo (Worker Nodes) al Detalle

Componentes en cada Servidor Worker

• **`kubelet`:** El agente principal del nodo. Se comunica con el API Server, recibe las especificaciones de los Pods (PodSpecs) y se asegura de que sus contenedores estén vivos y sanos.

• **`kube-proxy`:** El gestor de red del nodo. Mantiene las reglas de red e iptables/IPVS en el host para permitir que los Services balanceen tráfico hacia los Pods correctos.

• **Container Runtime (CRI):** El motor de bajo nivel responsable de descargar imágenes y ejecutar los contenedores reales (ej. `containerd` o `CRI-O`).

• **Pods:** La carga de trabajo real (tu aplicación Node.js, Python, PostgreSQL, etc.) ejecutándose dentro de contenedores.

$ Flujo Interno del Worker

```
# El flujo de ejecución en el Worker:
# 1. kube-scheduler asigna Pod al nodo 'k8s-worker-
01'
# 2. kubelet en k8s-worker-01 detecta la asignación
# 3. kubelet habla con containerd vía CRI (gRPC):
#    - PullImage(mi-app:1.0)
#    - CreateContainer()
#    - StartContainer()
# 4. kube-proxy actualiza reglas iptables en el
nodo
# 5. kubelet monitoriza periódicamente con liveness
probes
```

El Bucle de Reconciliación: Magia Declarativa

La Filosofía del Bucle Infinito

- **Estado Deseado (Desired State):** Lo que tú defines en los archivos `.yaml` (ej. 'Quiero 3 réplicas de la API').
- **Estado Actual (Current State):** Lo que realmente está corriendo en los servidores en este milisegundo.
- **El Ciclo de Reconciliación:** Los controladores de K8s ejecutan continuamente un bucle infinito: `OBSERVAR -> COMPARAR -> ACTUAR`.
- **Resiliencia Continua:** Si un contenedor muere por OOM o un kernel panic, el estado actual pasa a ser 2; el controlador detecta que 2 != 3 y arranca uno nuevo inmediatamente.

$ Lógica de Reconciliación

```
# Pseudocódigo del bucle de reconciliación:
while True:
    desired_state = read_from_etcd()
    current_state = inspect_actual_cluster()

    if current_state != desired_state:
        diff = calculate_difference(desired,
current)
        execute_remedy(diff) # Crear o matar Pods

    sleep(poll_interval)

# ¡Esto ocurre 24/7 sin intervención humana!
```

Primitiva Fundamental 1: El Pod

La Unidad Mínima de Cómputo

• **Kubernetes no ejecuta contenedores sueltos:** La unidad básica de despliegue y escalado siempre es el **Pod**.

• **Un grupo de 1 o más Contenedores:** Habitualmente aloja un único contenedor principal de aplicación, pero puede alojar contenedores auxiliares ('Sidecars').

• **Red Compartida:** Todos los contenedores de un Pod comparten el mismo Network Namespace: tienen la misma IP y se comunican entre sí por `localhost`.

• **Almacenamiento Compartido:** Los volúmenes montados en el Pod son accesibles por todos los contenedores que residen en su interior.

$ Ejemplo de Pod en YAML

```yaml
# Manifiesto YAML de un Pod simple (pod.yaml)
apiVersion: v1
kind: Pod
metadata:
  name: mi-pod-web
  labels:
    app: frontend
spec:
  containers:
  - name: nginx-server
    image: nginx:alpine
    ports:
    - containerPort: 80
    resources:
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Primitiva Fundamental 2: Deployments y Réplicas

Gestión de Ciclo de Vida y Réplicas

• **Jamás Desplegar Pods Sueltos en Producción:** Si un Pod suelto muere, nadie lo recrea. Siempre se gestionan mediante un **Deployment**.

• **Controlador de Réplicas (ReplicaSet):** Garantiza que siempre exista el número exacto de réplicas saludables especificadas en `spec.replicas`.

• **Estrategia RollingUpdate:** Actualiza los Pods de forma gradual (ej. 25% a la vez) asegurando que siempre hay réplicas sirviendo tráfico sin downtime.

• **Rollback Instantáneo:** Si la versión nueva falla, puedes volver a la versión anterior con un solo comando: `kubectl rollout undo deployment/mi-app`.

$ Ejemplo de Deployment

```
# Manifiesto YAML de un Deployment (deployment.yaml)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-pedidos
spec:
  replicas: 3
  selector:
    matchLabels:
      app: pedidos
  template:
    metadata:
      labels:
        app: pedidos
    spec:
      containers:
      - name: api
        image: mi-repo/pedidos:2.0
        ports:
        - containerPort: 3000
```

Primitiva Fundamental 3: Services y Balanceo L4

La Abstracción de Red Estable

• **El Problema:** Los Pods son efímeros y sus IPs cambian constantemente cada vez que se reinician o escalan.

• **La Solución (Service):** Proporciona una IP virtual fija (ClusterIP) y un nombre DNS interno estable (`api-pedidos.default.svc.cluster.local`).

• **Mecanismo de Selección por Labels:** El Service utiliza `spec.selector` para encontrar dinámicamente qué Pods coinciden con la etiqueta.

• **Tipos de Service:** `ClusterIP` (solo interno), `NodePort` (abre puerto 30000-32767 en todos los nodos), `LoadBalancer` (provisiona un balanceador en AWS/Azure/GCP).

$ Ejemplo de Service en YAML

```yaml
# Manifiesto YAML de un Service (service.yaml)
apiVersion: v1
kind: Service
metadata:
  name: servicio-pedidos
spec:
  type: ClusterIP
  selector:
    app: pedidos    # Conecta con los Pods
etiquetados
  ports:
  - port: 80         # Puerto del servicio
    targetPort: 3000 # Puerto del contenedor

# Cualquier Pod del clúster puede consumir la API
# haciendo una petición HTTP simple a:
# http://servicio-pedidos:80
```

Configuración y Persistencia: ConfigMaps, Secrets y PVCs

Configuración Externa

• **ConfigMaps:** Almacenan pares clave-valor de configuración no sensible desacoplados de la imagen del contenedor.

• **Secrets:** Almacenan contraseñas, certificados y tokens codificados en Base64 (o cifrados con KMS/Vault).

• **Inyección Flexible:** Ambos objetos se pueden inyectar en los Pods como variables de entorno (`env`) o montados como archivos en carpetas de solo lectura.

• **Inmutabilidad del Código:** La misma imagen de contenedor se despliega en Desarrollo, Staging y Producción cambiando únicamente el ConfigMap.

Persistencia: PVC y StorageClass

• **PersistentVolumeClaim (PVC):** Petición formal de almacenamiento que realiza el Pod (ej. 'Necesito 50GB en modo ReadWriteOnce').

• **PersistentVolume (PV):** El disco físico real aprovisionado en la nube (AWS EBS, Azure Managed Disk, Ceph, NFS).

• **StorageClass:** Mecanismo de aprovisionamiento dinámico: si el Pod pide un PVC, K8s contacta con la API cloud y crea el disco al vuelo.

• **StatefulSets:** El controlador especializado para cargas de trabajo con estado persistente (bases de datos con réplicas primario/réplica ordenadas).

De Docker a Kubernetes: Diccionario de Conceptos

Tabla de Equivalencias Conceptuales

• `docker run -d ...` --> Kubernetes **Pod** (ejecución básica) o **Deployment** (ejecución gestionada).

• `docker build -t mi-app .` --> El proceso de build sigue siendo **100% idéntico** (las imágenes OCI se consumen directamente en K8s).

• **`-p 8080:** 80` (Port mapping) --> Kubernetes **Service** (ClusterIP, NodePort o LoadBalancer con selector de etiquetas).

• `docker network create` --> **CNI Plugin** (red plana donde todos los Pods se ven entre sí por IP sin NAT) + **NetworkPolicies**.

• **`-v mi_vol:** /datos` (Named Volume) --> **PersistentVolumeClaim (PVC)** respaldado por un StorageClass del proveedor.

• `docker compose up` --> Colección de manifiestos YAML aplicados con `kubectl apply -f ./manifests/` o empaquetados con **Helm Charts**.

Entornos Locales para Aprender Kubernetes

Opciones para Practicar en tu Portátil

- **Docker Desktop Kubernetes:** La opción más fácil si usas Docker Desktop. Basta marcar una casilla en 'Settings > Kubernetes > Enable' y tienes un clúster de un nodo.
- **Kind (Kubernetes in Docker):** Ejecuta nodos de Kubernetes como contenedores Docker en tu host. Ultrarrápido, excelente para pruebas de CI/CD.
- **Minikube:** La herramienta histórica oficial de la comunidad K8s. Soporta múltiples hipervisores, drivers de Docker y addons (ingress, dashboard, métricas).
- **K3s (de Rancher/SUSE):** Distribución certificada ultraligera (<100MB) compilada en un solo binario, ideal para Edge, IoT y portátiles con recursos modestos.

$ Clúster con Kind

```
# Arrancar un clúster con Kind en 30 segundos:
$ kind create cluster --name curso-docker
Creating cluster "curso-docker" ...
 ✓ Ensuring node image (kindest/node:v1.29.2)
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
Set kubectl context to "kind-curso-docker"

# Verificar conexión:
$ kubectl cluster-info
Kubernetes control plane is running at
https://127.0.0.1:...

# Destruir el clúster de prueba:
$ kind delete cluster --name curso-docker
```

Laboratorio 5: Despliegue de App en Kubernetes

Objetivos y Pasos del Ejercicio Práctico

- **Paso 1:** Verificar el acceso al clúster local (`kubectl get nodes`, `kubectl cluster-info`).
- **Paso 2:** Redactar el manifiesto `deployment.yaml` para una aplicación web con 3 réplicas y límites de recursos de memoria y CPU.
- **Paso 3:** Aplicar el manifiesto declarativamente (`kubectl apply -f deployment.yaml`) y observar la creación concurrente de los Pods.
- **Paso 4:** Redactar y aplicar el manifiesto `service.yaml` exponiendo el puerto del servicio mediante `NodePort` o `ClusterIP`.
- **Paso 5:** Probar la conectividad y el balanceo de carga automático entre las 3 réplicas mediante peticiones repetidas.
- **Paso 6:** Simular la caída de un Pod con `kubectl delete pod <nombre>` y comprobar cómo el bucle de reconciliación levanta uno nuevo en segundos.
- **Paso 7:** Realizar un Rolling Update modificando la versión de la imagen en el Deployment y observar el proceso de actualización sin corte de servicio.

¡ENHORABUENA! CURSO COMPLETADO

20 Horas Lectivas • Docker Fundamentals
Has dominado: Motor OCI, Seguridad de Kernel, Redes, Persistencia, Cloud y Kubernetes.
¡El camino hacia Cloud Native continúa!